

Input-Output Subsystem



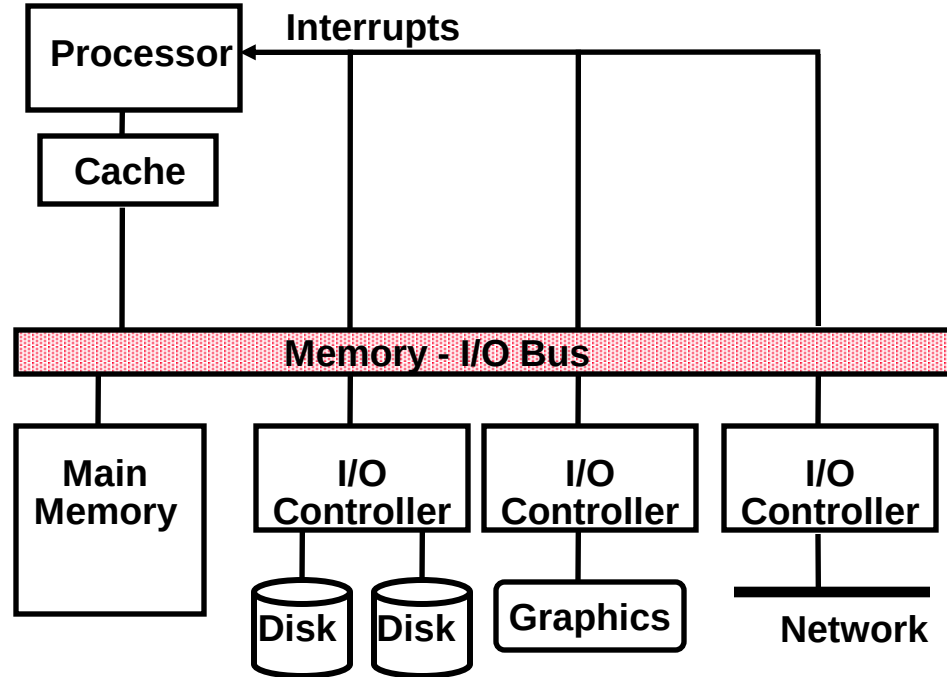
John Jose

Associate Professor

Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

Typical I/O Subsystem



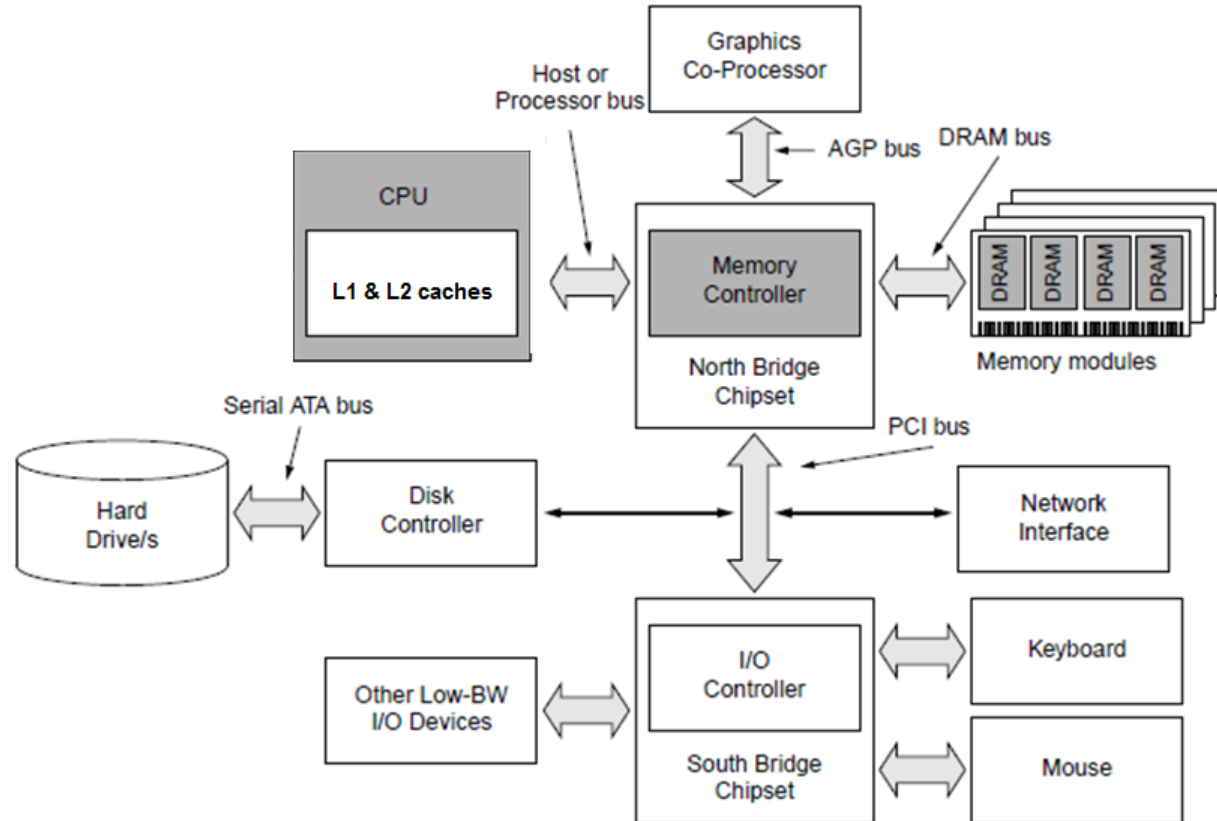
Problems in I/O Interfacing

- ❖ Wide variety of peripherals
 - ❖ Delivering different amounts of data
 - ❖ At different speeds
 - ❖ In different formats
- ❖ All slower than CPU and RAM
- ❖ Need I/O modules with some intelligence

Components of I/O Subsystem

- ❖ I/O Hardware
 - ❖ Ports, buses, devices, controllers
- ❖ I/O Software
 - ❖ Interrupt Handlers
 - ❖ Device Driver
 - ❖ Device-Independent Software
 - ❖ User-Space I/O Software
- ❖ I/O Data transfer mechanisms
 - ❖ Polling, Interrupt and DMAs

Where is I/O controller residing?

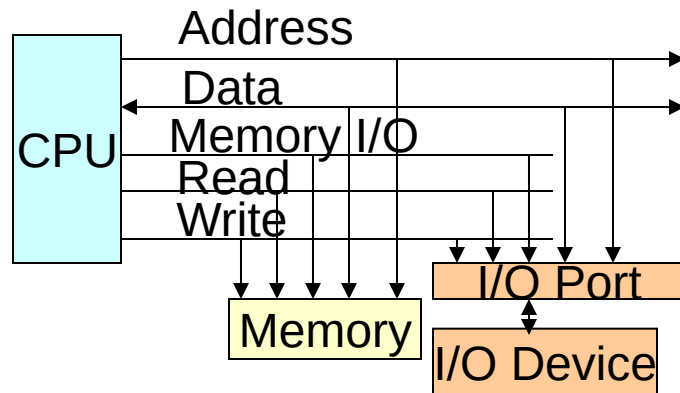
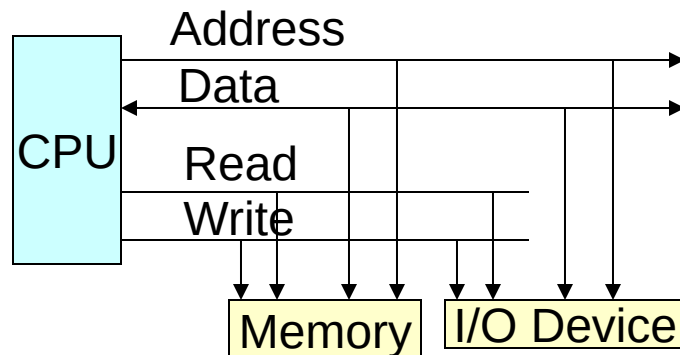


I/O Mapping

- ❖ Memory mapped I/O
 - ❖ Devices and memory share an address space
 - ❖ I/O looks just like memory read/write
 - ❖ No special commands for I/O
 - ❖ Large selection of memory access commands available
- ❖ Isolated I/O (I/O mapped I/O)
 - ❖ Separate address spaces
 - ❖ Need I/O or memory select lines
 - ❖ Special commands for I/O; Limited set

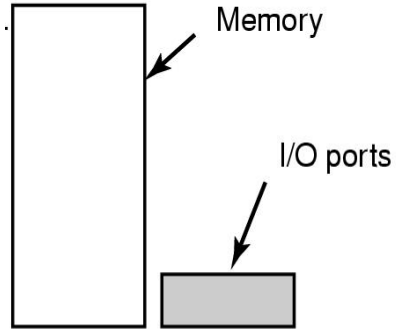
I/O Mapping

- ❖ CPU needs to talk to I/O
- ❖ **Memory mapped I/O**
 - ❖ Devices mapped to reserved memory locations - like RAM
 - ❖ Uses load/store instructions just like accesses to memory
- ❖ **I/O mapped I/O**
 - ❖ Special bus line
 - ❖ Special instructions



I/O Mapping

Two address



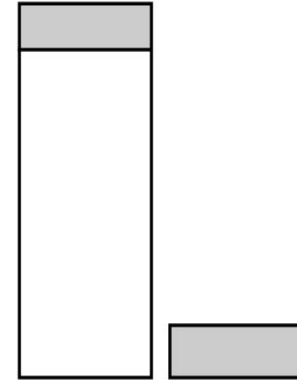
(a)

One address space



(b)

Two address spaces



(c)

(a) Separate memory & I/O address space

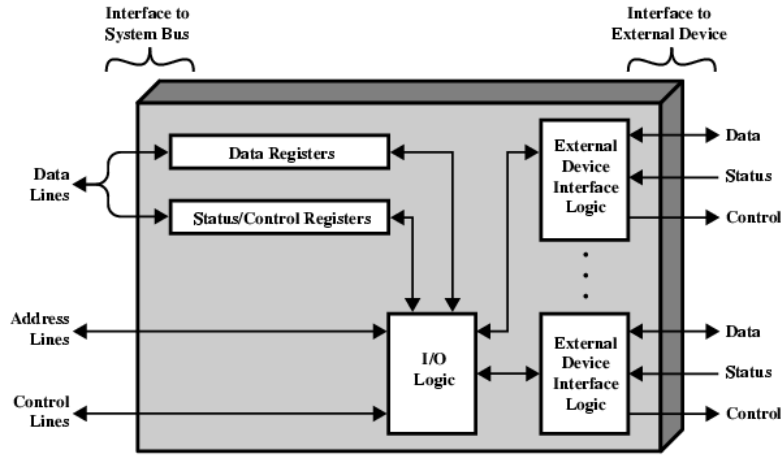
(b) Memory-mapped I/O

(c) both

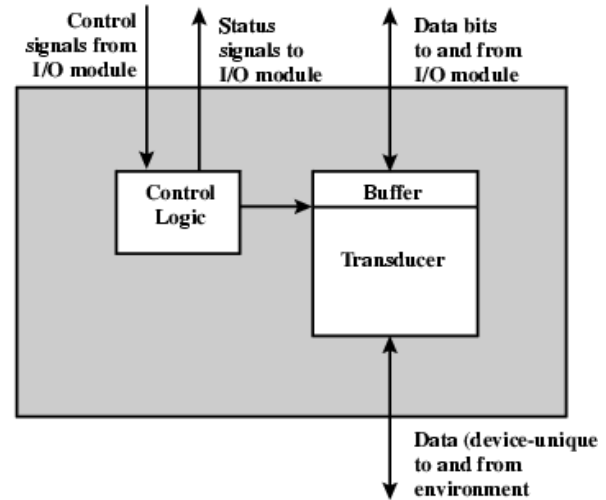
I/O Controller and Device Interfacing

- ❖ I/O module interface I/O to CPU and Memory
- ❖ **I/O controller $\leftrightarrow$ I/O devices ports**
 - ❖ Transfers data to/from device
 - ❖ Synchronizes operations with software
- ❖ **Status/ control registers:** device status, errors
- ❖ **Data registers**
 - ❖ Write: CPU/RAM data $\rightarrow$ device [Transmit]
 - ❖ Read: CPU $\leftarrow$ device [Receive]

I/O Controller and Device Interfacing



I/O Controller Module



External Device Module

Functions of I/O Controller Module

- ❖ Control & Timing
- ❖ Processor Communication
- ❖ Device Communication
- ❖ Data Buffering
- ❖ Error Detection and Correction

Basic I/O Steps

- ❖ CPU checks I/O module device status
- ❖ I/O module returns status
- ❖ If ready, CPU requests data transfer by sending a command to the I/O module
- ❖ I/O module gets a unit of data (byte, word, etc.) from device
- ❖ I/O module transfers data to CPU

I/O Data Transfer techniques

- ❖ **Programmed I/O**
- ❖ **Interrupt-Driven I/O**
- ❖ **Direct Memory Access (DMA)**

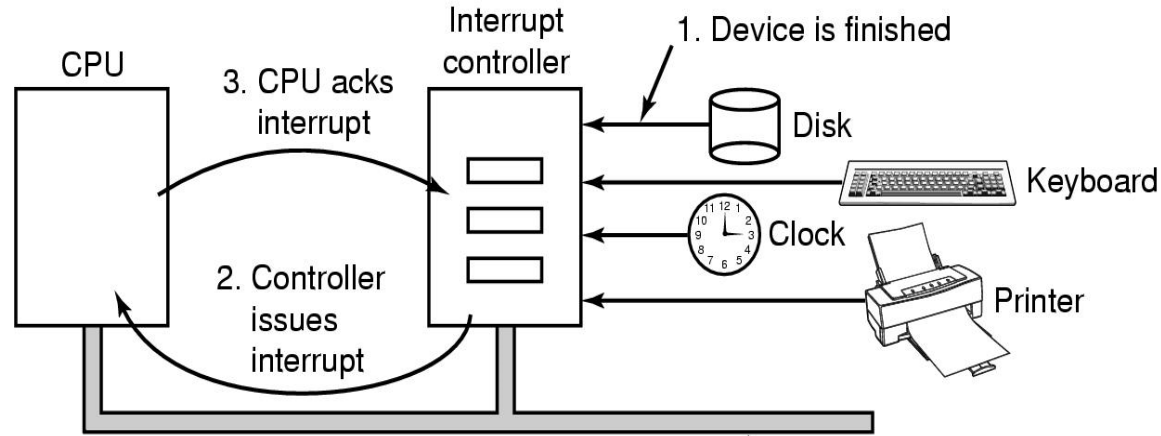
Programmed I/O

- ❖ CPU periodically check I/O status (polling)
 - ❖ If device ready, do operation
 - ❖ If error, take action
- ❖ CPU has direct control over I/O
 - ❖ Sensing status
 - ❖ Read/write commands
 - ❖ Transferring data
- ❖ CPU waits for I/O module to complete operation
- ❖ Wastes CPU time

Steps in Programmed I/O

- ❖ CPU requests I/O operation
- ❖ I/O module performs operation
- ❖ I/O module sets status bits
- ❖ CPU checks status bits periodically (polling)
- ❖ CPU may wait or come back later
- ❖ I/O module does not inform CPU directly
- ❖ I/O module does not interrupt CPU

Interrupt Driven I/O



- ❖ I/O module gets data from peripheral while CPU continues other work.
- ❖ I/O module interrupts CPU when data is ready.
- ❖ CPU requests data
- ❖ I/O module transfers data

Interrupt Driven I/O

- ❖ I/O device issues an interrupt to indicate that it needs attention of CPU
- ❖ Interrupts are special signals initiated by I/O devices to catch the attention of the processor.
- ❖ Overcomes CPU waiting
- ❖ No repeated CPU checking of device
- ❖ An I/O interrupt is **asynchronous** w.r.t. instruction execution
- ❖ Is not associated with any instruction so doesn't prevent any instruction from completing

Interrupt Driven I/O

- ❖ Advantages of using interrupts

- ❖ Relieves the processor from having to continuously poll for an I/O event; user program progress is only suspended during the actual transfer of I/O data to/from user memory space

- ❖ Disadvantage – special hardware is needed to

- ❖ Indicate the I/O device causing the interrupt and to save the necessary information prior to servicing the interrupt and to resume normal processing after servicing the interrupt

Challenges in Interrupt Driven I/O

❖ How do you identify the module issuing the interrupt?

- ❖ Need a way to identify the device generating the interrupt

- ❖ Vectored vs Non-Vectored Interrupts

❖ How do you deal with multiple interrupts?

- ❖ Can have different urgencies (so need a way to prioritize them)

- ❖ Maskable vs Non-Maskable Interrupts

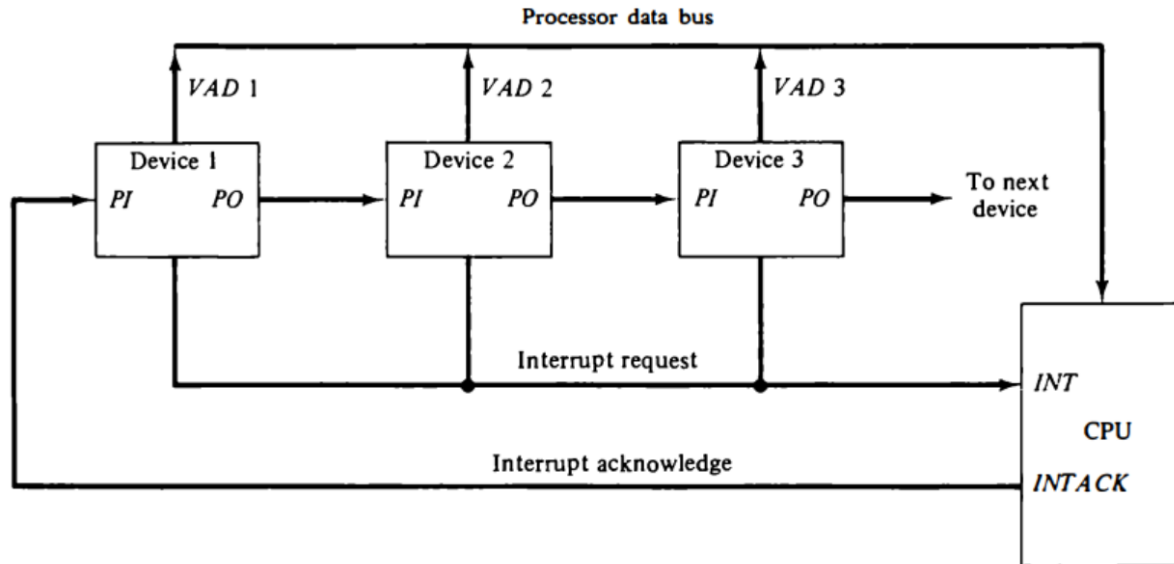
- ❖ Interrupt Priority

Identifying Interrupting Module

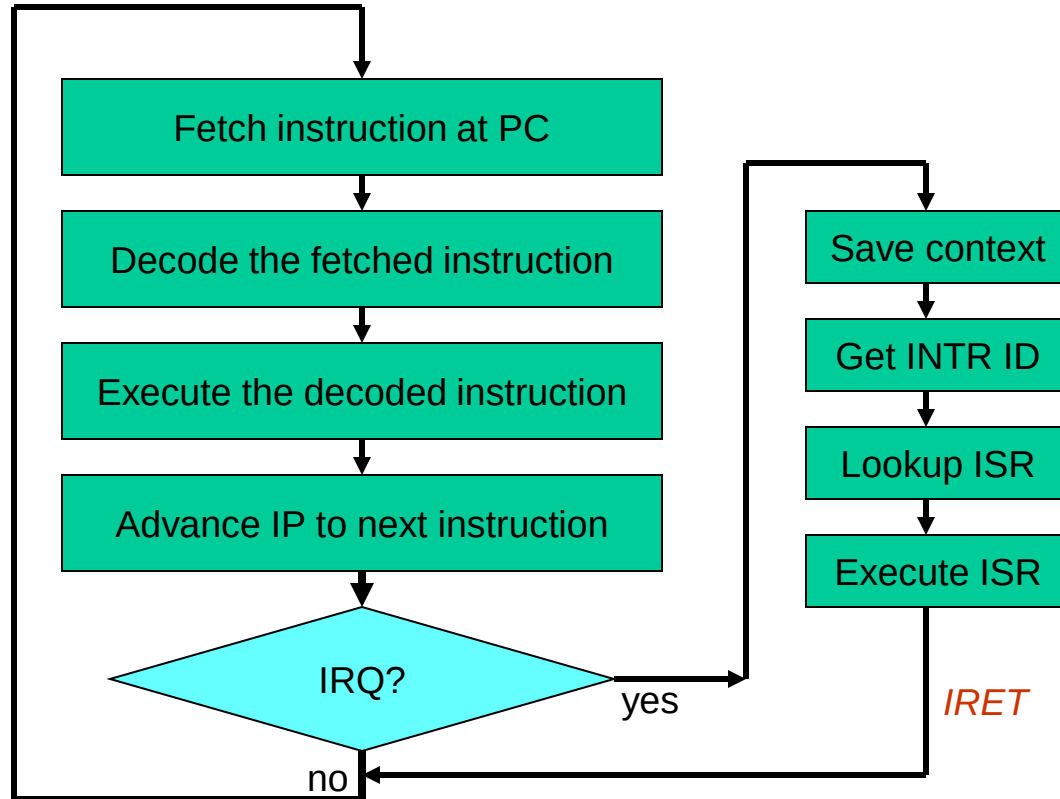
- ❖ Different line for each module
 - ❖ Limits number of devices
- ❖ Bus Master
 - ❖ Module must claim the bus before it can raise interrupt

Identifying Interrupting Module

- ❖ CPU asks each module in turn (Slow) Daisy Chain or Hardware poll
 - ❖ Interrupt Acknowledge sent down a chain
 - ❖ Module responsible places vector on bus
 - ❖ CPU uses vector to identify handler routine



Interrupt Service Routine



Direct Memory Access

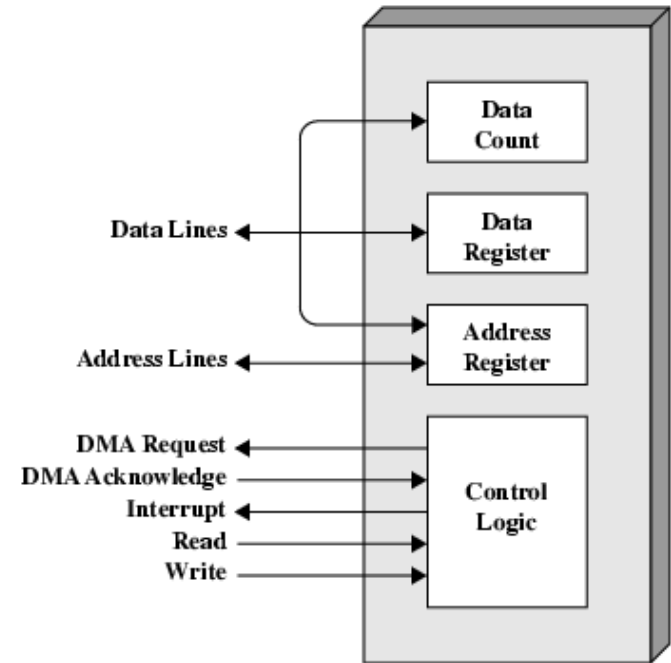
- ❖ Interrupt driven and programmed I/O require active CPU intervention
- ❖ For high-bandwidth devices (like disks) interrupt-driven I/O would consume a lot of processor cycles
- ❖ DMA is an additional module (hardware) on bus
- ❖ DMA controller takes over from CPU for I/O operations

Direct Memory Access

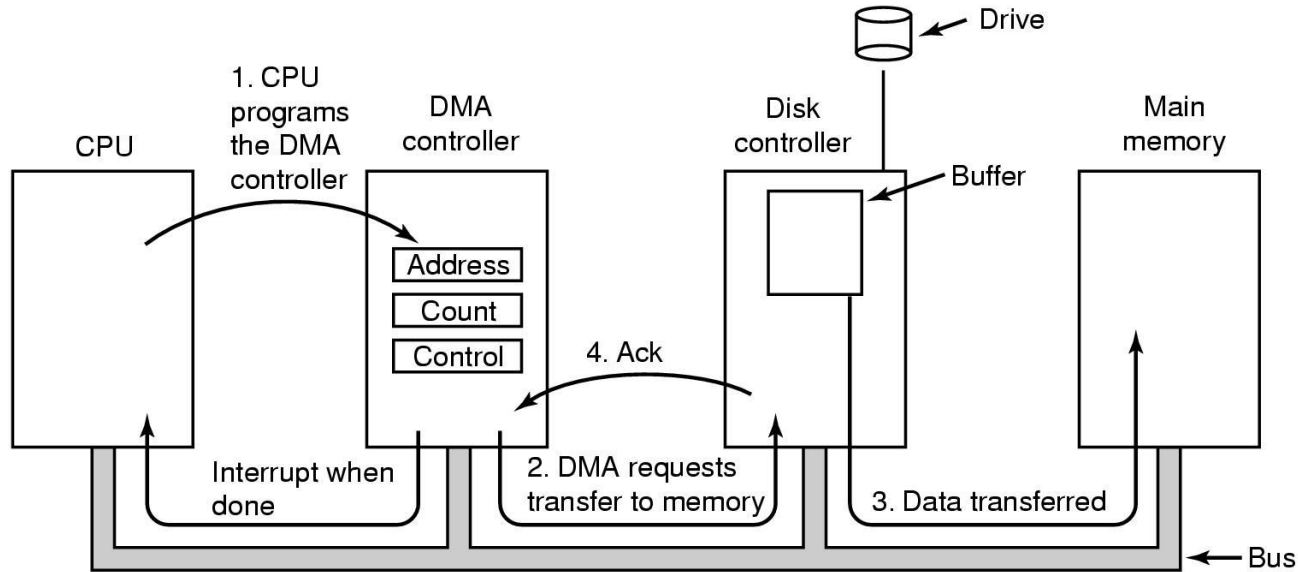
- ❖ DMA controller has the ability to transfer large blocks of data directly to/from the memory without involving the processor.
- ❖ The processor initiates the DMA transfer by supplying the I/O device address, the operation to be performed, the memory address destination/source, the number of bytes to transfer
- ❖ The DMA controller manages the entire transfer (possibly thousand of bytes in length), arbitrating for the bus
- ❖ When the DMA transfer is complete, the DMA controller interrupts the processor to let it know that the transfer is complete

DMA Module Diagram

- ❖ CPU tells DMA controller:-
 - ❖ Read/Write
 - ❖ Device address
 - ❖ Starting address of memory block for data
 - ❖ Amount of data to be transferred
- ❖ CPU carries on with other work
- ❖ DMA controller deals with transfer
- ❖ DMA controller sends interrupt when finished



Disk to Memory Copy via DMA



Modes of DMA operation

❖ Cycle Stealing

- ❖ DMA controller acquires control of bus
- ❖ Transfers a single word and releases the bus
- ❖ The CPU is slowed down due to bus contention
- ❖ Responsive but not very efficient

❖ Burst Mode

- ❖ DMA Controller acquires control of bus
- ❖ Transfers all the data and then only releases the bus
- ❖ The CPU is suspended or works with cache
- ❖ Efficient but interrupts may not be serviced in a timely way



johnjose@iitg.ac.in
<http://www.iitg.ac.in/johnjose/>